# 有限位元中的算術契約

Computer arithmetic 是 finite-width representation 與 operation semantics。硬體加法器可以處理 bit patterns，但「候選結果超出可表示範圍」要由 signed interpretation、instruction 與 status flags 決定如何處理。

本章分三條線：integer ALU、multiply/divide state machine、IEEE 754 floating point。每條線都要指定 bit width、rounding、exception 與 timing model。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 算術章的問題

固定 32-bit word 無法表示所有整數、分數或實數。Overflow、underflow、rounding 與特殊值不是書寫誤差，而是 representation 契約的一部分。

硬體設計要平衡 latency、throughput、面積與 exception precision。Fast multiplier 可以用更多 partial-product hardware 換取較少 iterations；IEEE add 可用 extra bits 降低 rounding error，但增加 pipeline state。

本章先定義 operation result，再設計 hardware。不能只從 gate diagram 反推語意。

例如 8-bit unsigned 乘法的 operands 各 8 bits，完整 result 需 16 bits；若只存低 8 bits，即明定取 modulo 256。Binary32 加法則輸入 24-bit 有效精度的值，先形成 exact sum，再在規定 rounding point 映回 binary32；兩種 result 規則不可混用。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 3.1 算術章的基本模型

固定 n-bit 的加減法若只保留低 n bits，就是模 $2^n$ 運算；除法另有商與餘數的規則，不能一概套用這個模型。遇到溢位時，指令也可能選擇 trap 或飽和結果。Floating point 則在有限 set $F$ 上操作，先依 exact 或 extended-precision rule 計算，再把無法精確表示的結果映射到 $F$。

對任何 operation $\circ$，硬體要給出

$$y = \mathrm{round}(x \circ z),$$

其中 rounding function、exception conditions 與 special cases 都必須明確。

Floating point 與 real arithmetic 不同。運算結果滿足 ISA 定義的 correctly rounded behavior，不代表在所有平台上都得到無限精確的數學結果。

整數 `0/0` 的處理依 ISA 或語言而異；IEEE 浮點 `0/0` 為 invalid，通常產生 quiet NaN。先指定型別與架構再談除法。

## 3.2 二補數加減

對 $n$-bit two's complement integers，bit vector 可解釋為

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

Addition 的 bit-level result 與 unsigned addition 相同，輸出低 $n$ bits。Signed value 是否等於 mathematical sum 取決於是否 overflow。

Subtraction 可寫成 $a - b = a + \bar{b} + 1$，因此可用相同 adder 與 $carry_in = 1$。此代數關係是硬體共用 adder 的基礎。

Modular bit pattern 正確不等於 signed mathematical result 可表示。兩者要分開陳述。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Signed overflow 的推導

令 operands 與 result 的 sign bits 分別為 $s_a, s_b, s_r$。Addition 只在 operands 同號且 result 異號時 signed overflow：

$$ov = \overline{s_a \oplus s_b} \wedge (s_a \oplus s_r).$$

等價地，carry into MSB 與 carry out of MSB 不同，也代表 signed overflow。

Unsigned overflow 則只由 carry-out 表示。比較 $0xffffffff$ 與 1 時，unsigned 是最大值對 1；signed 是 -1 對 1。

用 `0x7fffffff+1` 和 `0xffffffff+1` 分別檢驗 signed 與 unsigned 條件。兩者不可共用一個「overflow 等於 carry」的句子。

### Addition/subtraction 的位元級驗算

令 $c_7$ 是進 sign bit 的 carry，$c_8$ 是離開 sign bit 的 carry。對 8-bit addition，signed overflow 等價於

$$ov = c_7 \oplus c_8.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Signed overflow 的推導 （續）

| Operation | Operand bits | Result bits | $c_7$ | $c_8$ | Unsigned carry-out | Signed overflow |
|---|---|---|---|---|---|---|
| `0x7f+0x01` | `01111111+00000001` | `10000000` | 1 | 0 | 0 | 1 |
| `0x80+0xff` | `10000000+11111111` | `01111111` | 0 | 1 | 1 | 1 |
| `0x80-0x01` | `10000000+11111110+1` 的減法模型 | `01111111` | 0 | 1 | 1 | 1 |
| `0xff+0x01` | `11111111+00000001` | `00000000` | 1 | 1 | 1 | 0 |

`0xff+1` 的 $c_7 = c_8 = 1$，只表示 unsigned carry-out。相同結果 bits 若由 `slt/sltu` 解讀，大小順序也不同。減法可改寫為 $a - b = a + \bar{b} + 1$；對 subtraction，$ov = (s_a \oplus s_b) \wedge (s_a \oplus s_r)$。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 加減法時序與例外

Simple ripple-carry adder 的 carry 依序傳遞，critical path 隨 width 成長。Carry-lookahead 與其他 parallel-prefix 結構用更多 gates 縮短 carry path，但增加面積與 wiring。

若 instruction 是 `add`，signed overflow 可能寫入例外原因並阻止 architectural result commit。`addu` 採 modulo result，不因同樣條件 trap。具體的 exception 與 register side effect 以 ISA 定義為準。

Timing 必須以最壞可成立 path 與 setup/hold constraints 決定，不能只用平均輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 多媒體飽和運算

Saturated arithmetic 在超出 representable range 時把結果夾到最小或最大值。例如 signed 8-bit 的 $100 + 100$ 飽和結果是 127，而不是 modular bit pattern。

Saturation 避免 wrap，但會損失可逆性。Encryption、checksum 或需要 exact modulo 的程式不適合任意改用飽和。

SIMD packed arithmetic 可同時處理多個短 integers。Carry 不跨 lane，lane width 與 saturation mode 是 instruction 契約的一部分。

同為 signed 8-bit 的 100+100，audio saturation 夾為 127；保留低 8 bits 則為 0xc8，作 two’s-complement 解讀是 −56。若以 unsigned byte 解讀 0xc8 則是 200。選擇 saturate、wrap 及 interpretation 會改變可觀察結果。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 3.3 乘法硬體

Unsigned $n \times n$ multiplication 的 exact product 最多需 $2n$ bits。32-bit operands 的 exact result 是 64 bits，結果不應先截成 32 bits 再計算。

Shift-and-add 依據

$$A \times B = \sum_{k=0}^{n-1} b_k (A \ll k).$$

若 $b_k = 1$，累加 shifted multiplicand；若 $b_k = 0$，不需要累加。乘 2 的冪可由移位完成，但生成一般 partial products 仍需 AND gates。

先固定是 signed 還是 unsigned multiplication。相同 bit pattern 在兩種 interpretation 會得到不同 mathematical value。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 循序乘法時序與正確性

對 unsigned $n$-bit $M$、$Q$，每回合只看 $Q_0$，需要時計算

$$C : A \leftarrow A + M,$$

再把 $C, A, Q$ 串成 $2n+1$ bits 右移一位；$C$ 進入 A 的 MSB，舊 $Q_0$ 已處理並移出；舊 $Q_1$ 移入新的 $Q_0$，才是下回合 decision bit。下表以 $M = 1101_2 = 13$、$Q = 1011_2 = 11$ 明定 A 為 shift 後的 4-bit 值，C 只表示該回合加法的第五個 carry bit。

| Round | $Q_0$ | Add result $C : A$ | Shift 後 A | Shift 後 Q |
|---|---|---|---|---|
| 0 | | | 0000 | 1011 |
| 1 | 1 | 0:1101 | 0110 | 1101 |
| 2 | 1 | 1:0011 | 1001 | 1110 |
| 3 | 0 | 0:1001 | 0100 | 1111 |
| 4 | 1 | 1:0001 | 1000 | 1111 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 循序乘法時序與正確性 （續）

最終 (A,Q)=10001111₂=143=13×11。要證明右移程序，固定初始multiplier為 $q$，完成 $k$ 回合的暫存器為 $A_k, Q_k$，定義 $R_k = 2^n A_k + Q_k$。精確不變量是

$$R_k = \left\lfloor \frac{q}{2^k} \right\rfloor + M(q \bmod 2^k)\, 2^{n-k}, \qquad 0 \le k \le n.$$

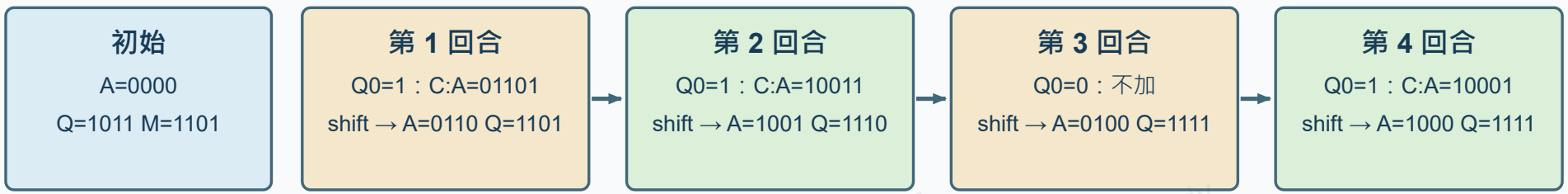
$k = 0$ 時 $R_0 = q$，成立。若 $k < n$，右項第二項為偶數，所以當前 $Q_k$ 的最低位恰為原multiplier第 $k$ 位 $q_k$。依此位決定加法後，完整 $C : A : Q$ 的值為 $R_k + q_k M 2^n$；右移得到

$$R_{k+1} = \left\lfloor \frac{R_k + q_k M 2^n}{2} \right\rfloor = \left\lfloor \frac{q}{2^{k+1}} \right\rfloor + M(q \bmod 2^{k+1}) 2^{n-k-1}.$$

這使用 $q \bmod 2^{k+1} = (q \bmod 2^k) + q_k 2^k$，因此歸納延續。$k = n$ 時因 $q < 2^n$，有 $R_n = Mq$；A為乘積高半部，Q為低半部。Carry C保留本回合加法的溢位並隨右移進A，沒有遺失該位。13×11的實際 $R_k$ 依序為 11、109、158、79、143。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

### 13 × 11 的 4-bit shift-and-add 狀態




**最後乘積：(A,Q)=$10001111_2$ = 143 = 0x008f。**

C 是加法 carry；右移時 C 進 A；舊 Q0 已處理並移出，舊 Q1 進入新 Q0 作下一回合決策。

成本模型先寫清楚才可比較：

$$T_{\text{serial}} = t_{\text{setup}} + n\,t_{\text{iter}} + t_{\text{finalize}}.$$

若每回合的 add 與 shift 能同 cycle 完成，32-bit model 是 32 iterations；若它們各需一個 cycle，不能仍宣稱固定 32 cycles。Parallel tree 減少 dependent additions，但增加 partial-product wires、compressors 與 power；pipelined multiplier 可縮小 initiation interval，不會縮短單筆 latency。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 有號乘法

一種方法是記錄 operands 的 signs，取 magnitudes 作 unsigned multiplication，再依 sign product 決定結果符號。

對 $n$-bit signed operands，exact product 的 signed range 需要 $2n$ bits。最極端負值平方可能需要完整的 $2n$-bit pattern。

另一種硬體方法使用 Booth recoding，直接從 two's complement bits 生成 signed partial products，避免先取 absolute value。Radix-4 Booth 每次看多個 bits，減少 partial products，但 control 與 sign extension 更複雜。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 乘法加速

Parallel multiplier 一次產生多個 partial products，再用 Wallace tree、compression tree 或 carry-save adders 壓縮。Carry-save 可避免每一階段都傳遞長 carry chain。

Final carry-propagate adder 通常在最後產生 non-redundant result。樹狀結構降低 latency，代價是較多 full adders、routing 與 power。

Pipeline multiplier 可接受新 operand 的間隔比單次結果 latency 更短。Throughput 與 latency 是不同指標。

如果每 cycle 發一個 multiply 但結果三 cycles 後才可用，需 register scoreboard 或 hazard detection。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# MIPS 乘法指令

MIPS 使用 `mult`、`multu` 把 product 放入 64-bit HI/LO pair。`mfhi`、`mflo` 把指定 half 搬到 general-purpose register。

Unsigned 與 signed instruction 的 bit-level hardware 可共用，但高半部的數值意義不同；傳統 MIPS32 `mult`/`multu` 不因乘積超出 32 bits 而 trap，兩者都產生完整 HI/LO 結果。

若程式只取 32-bit low product，high bits 的 overflow 可能未被檢查。決定 result 是否精確要看成員需求與 instruction semantics。

不要把 `multu` 解讀為 operands 永遠是正數；它是 unsigned interpretation of bit patterns。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.4 除法硬體

Division 求 quotient $q$ 與 remainder $r$，使

$$D = Q \times d + r, \qquad 0 \le r < |d|.$$

這個範圍版本適用 unsigned division。Signed division 還要指定 quotient 的 truncation direction 與 remainder sign；不同語言、ISA 的代數約定可能不同。

Divide by zero 與 `INT_MIN / -1` 的結果都必須由具體 ISA 定義。傳統 MIPS32 `div`/`divu` 不因除以零或有號除法溢位而產生算術 trap；這些情形的 HI/LO 結果不可依賴，程式需先檢查。其他 ISA 或語言可能定義 trap，不能混用。

四條必要檢查是 divisor zero、sign policy、rounding direction、minimum-value boundary。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Restoring division algorithm

無號 restoring division 每回合先把餘數與商暫存器合併左移，再以餘數減除數。候選差非負就保留，並把本回合商位設為 1；候選差為負則加回除數，恢復左移後的餘數，商位設為 0。

接下來固定 13÷5 的 4-bit 模型：商暫存器 Q 初值為 1101，餘數 A 初值為 0，除數 M=5。A 使用 5 bits 保留試減的符號；每回合完成後須保持 0≤A<M。四回合的寄存器狀態與一般不變量放在下一節，避免混用不同輸入或不同迭代次數。

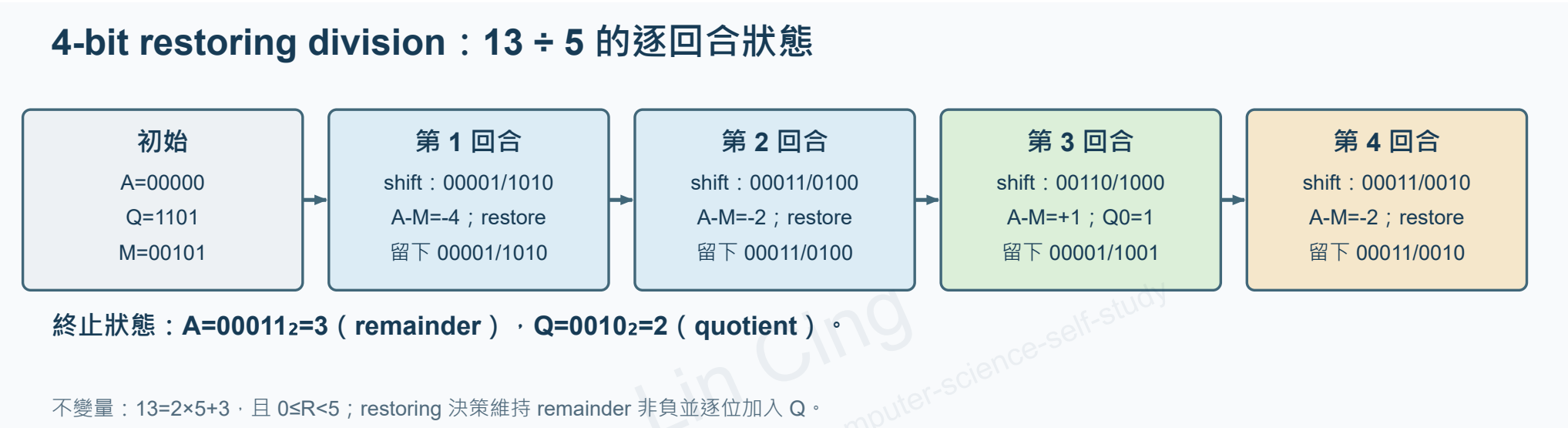
## Restoring division 的逐位不變量

用 bitwise long division 描述較容易證明。第 $k$ 回合開始時，令已產生 quotient bits 為 $Q_k$、目前 remainder 為 $A_k$，並維持

$$P_k = Q_k M + A_k,$$

其中 $P_k$ 是已讀入的 dividend prefix。下一 dividend bit $d$ 到來後，候選 remainder 是 $2A_k + d$。若它至少 $M$，本回合 quotient bit 放 1，remainder 改為 $2A_k + d - M$；否則放 0，remainder 保持 $2A_k + d$。兩個分支都維持上一式，因此做完所有 dividend bits 後 $D = QM + A$，且 $0 \le A < M$。這就是 restoring division 的正確性。

# Restoring division algorithm （續）




對 $D = 13$、$M = 5$，最終 $A = 3$、$Q = 2$。四個 shifts 後驗證 $13 = 2 \times 5 + 3$。這個 combined-register variant 有 $n$ rounds；書上另一個不提前折疊 register 的 hardware variant 可能寫成 $n + 1$ rounds，兩者 iteration 數不能混用，卻必須得到同一 pair。

## Signed division 與 faster division

取 magnitudes 後作 unsigned division，再依 signs 決定 quotient sign。Remainder sign 通常跟 dividend 一致，因為 $D = Qd + R$；實際規則仍依 ISA 或 programming language 定義。

SRT divider 每回合用 estimate 與 residual recurrence 產生多個 quotient bits。Radix-4 每回合可產生 2 bits，重複次數約為 binary radix 的一半，但需要 quotient-digit selection table。

對極端 divisor、leading-zero 分佈與 bit pattern 的高度偏差，可做 operand normalization 與 early termination。這屬實作最佳化，不可改變 architecture-visible result。

不要把 radix-4 的「2 bits per iteration」直接當成精確 2 倍 speedup。Overhead、normalization 與 correction cycles 仍存在。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Divide 指令

教材的 MIPS 使用 `div`、`divu`，結果分布在 HI/LO。程式再用 `mfhi` 取 remainder、`mflo` 取 quotient。

Divide latency 常較 multiply 長，且 fewer iterations 不代表 pipeline throughput 同比例改善。若 compiler 看到除數是常數，可能用 multiply by reciprocal 或 shifts，但必須證明 rounding semantics 等價。

Optimization 只在輸入範圍與 compiler 支援的 rounding model 相符時有效。

以 two's-complement `x=-7` 為例，MIPS `sra` 右移1得到 -4（朝負無窮），而 truncation-toward-zero 的有號除以2得到 -3。對此一位右移可先為負數加1，再 `sra` 得 -3；一般除以2的k次方須先證明適當 bias、位寬及不溢出，不能把任意 shift 直接當除法。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 3.5 IEEE 754

IEEE 754 binary floating-point 以 sign、biased exponent 與 fraction 表示

$$value = (-1)^s \times \text{significand} \times 2^e.$$

Normal binary32 有 1 sign bit、8 exponent bits、23 fraction bits。Exponent field $1$ 至 $254$ 對應 biased exponent，bias 為 127，leading 1 隱含在 significand。

Binary64 有 1、11、52 bits，bias 為 1023。Exponent width 和 significand precision 不能互相替代。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 浮點欄位

## IEEE 754 binary32 欄位與 -12.5 編碼




**0xC1480000**

sign=1；exponent=$10000010_2$

-12.5 = $-1.1001_2 \times 2^3$

**value = (-1)^s × 1.fraction × 2^(exponent-127)**

Exponent 全 0 改用 subnormal 規則；全 1 表示 infinity 或 NaN，不能一律套 hidden 1。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Normal number 的 value 可寫成

$$(-1)^s (1.F)_2 \, 2^{E_{\text{field}} - bias}.$$

其中 $F$ 是 fraction bits。Single 的 significand precision 是 $23 + 1 = 24$ bits，double 是 $52 + 1 = 53$ bits。

# 浮點欄位 （續）

Denormal/subnormal 在 exponent field 為零時使用，leading bit 改為 0，single 的 implicit exponent 是 $-126$。它讓零附近保持 gradual underflow。

`hidden 1` 不是永遠存在。Exponent field 為 zero 時必須切換 significand rule。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 特殊值與例外

Exponent field 全為 1 時表示 infinity 或 NaN。Fraction 為 0 是 signed infinity；fraction 非 0 是 NaN。NaN 又分 quiet 與 signaling，payload 與 propagation 有 IEEE 規則。

Operations 可能設定 overflow、underflow、inexact、invalid 與 divide-by-zero flags。Trap 是否啟動由 control register 與程式語言環境決定。

$+\infty + (-\infty)$ 是 invalid operation，通常產生 quiet NaN。$1/+\infty$ 在 IEEE arithmetic 中可以是 $+0$，與 integer divide by zero 不同。

不要用程式語言的 exception 名稱替代 IEEE flag 名稱；兩者的 trap 機制與 observable 行為可能不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 浮點加法：先對齊，再決定捨入

有限正常值相加時，先比較指數，把指數較小的有效數（significand）右移到共同尺度；異號輸入改作大小相減，再由較大者決定符號。移出的低位仍須保留資訊，不能先截斷。

完成加減後，先正規化，再依目標精度捨入；若捨入產生新的進位，還要再正規化並調整指數。特殊值、零、溢位與 subnormal 路徑須另外處理。

以下使用 **p=4 個有效位（含首位 1）、RNE、指數範圍不設限** 的縮小模型，讓所有中間位元可見；它不是 binary32 格式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 對齊數例：sticky bit 如何改變結果

令 a=1、b=9/128=0.0703125。b 原為 $1.001_2 \times 2^{-4}$，對齊到 $2^0$ 後為 $0.0001001_2$；兩個輸入在 p=4 都是精確值。

**p=4（含首位 1），RNE；指數範圍不設限**

| ×$2^0$ | 保留 4 bits | G | R | 其餘 bits |
|---|---|---|---|---|
| a = 1 | 1.000 | 0 | 0 | 00 |
| b = 9/128 | 0.000 | 1 | 0 | 01 |
| **a + b** | **1.000** | **1** | **0** | **01** |

$b = 1.001_2 \times 2^{-4} = 0.0001001_2 \times 2^0$

**S = OR(0,1) = 1 → GRS = 101 → $1.001_2 = 1.125$**

若把 tail 當 00，會誤判為 tie，得到 $1.000_2$。

精確和為 137/128；相鄰候選是 1 與 9/8，中點為 17/16=136/128。精確和大於中點，故選 9/8，誤差 7/128。若先丟棄低位而回傳 1，誤差是 9/128，已大於半個 ULP=1/16。

# 相消後的正規化：先找回首位 1

同一個 p=4 模型計算 0.75−0.6875。輸入指數相同，不必先移位；相減後卻要左移三位，才能恢復 normal 的首位 1。

**p=4、RNE：0.75 − 0.6875，兩個輸入皆可精確表示**

| | | |
|---|---|---|
| a | **1.100 000** | × $2^{-1}$ |
| − b | **1.011 000** | × $2^{-1}$ |
| 相減 | **0.001 000** | × $2^{-1}$ |
| 左移 3 位 | **1.000 000** | × $2^{-4}$ |

每左移一位，指數減一；數值維持不變。

GRS=000，結果 1/16=0.0625 是精確值，inexact=0。

本例的輸入與差都精確，沒有捨入誤差。相消可能放大**輸入先前已有的相對誤差**，不代表每一次相減本身都不精確。若左移時已碰到最小 normal 指數，必須改走 subnormal 路徑，不能無限降低指數。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## Guard、round、sticky：從距離推導 RNE

先看結果的大小。最低保留位記為 L；其後第一位為 guard G，第二位為 round R，更右所有位元取 OR 得 sticky S。S 是「後方是否有非零位」的摘要，不是單一位元的位置。

相對一個保留單位 ULP，G 的權重是 1/2，R 是 1/4，其餘尾端小於 1/4。故 G=0 時低於中點；G=1、R=S=0 時恰在中點；G=1 且 R 或 S 非零時高於中點。中點選最低保留位為 0 的候選，於是

$$\mathrm{inc} = G \wedge (R \vee S \vee L).$$

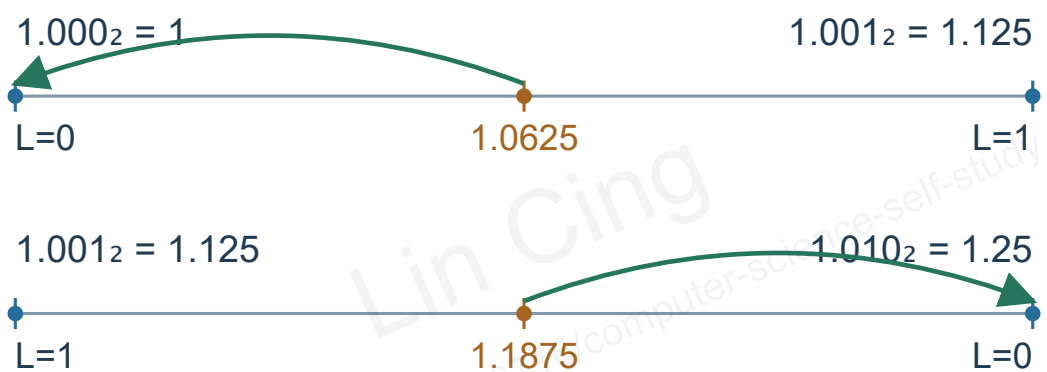
inc=1 時對**保留的大小**加一個 ULP，之後加回符號。這對負數可能使數值更負，因此不能一律稱為「向上捨入」。只要丟棄部分非零，結果就不等於精確值，須記錄 inexact。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 同樣是中點，為何有時保留、有時加一？

下圖仍用 p=4。第一個區間左端為偶，第二個區間右端為偶；even 指有效數整數編碼的最低位是 0，不是十進位值是否為偶數。

**p=4、RNE；每條數線只畫一個相鄰區間**




**inc = G ∧ (R ∨ S ∨ L)**

負數先捨入大小，再加回符號：−1.1875 → −1.25。

捨入也可能產生進位。例如 $1.1111_2$ 截成 p=4 後 L=1、GRS=100，須加一：$1.111_2+0.001_2=10.000_2$，再正規化為 $1.000_2\times2^1$。不能只修改 fraction 欄而漏掉指數增加。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Binary32 的兩個捨入檢查點

binary32 在 [1,2) 的間距為 $2^{-23}$，精度 p=24。對 $1+2^{-24}$，L=0、GRS=100，故選 1（0x3f800000）；對 $1+3\cdot 2^{-25}$，GRS=110，故選 $1+2^{-23}$（0x3f800001）。兩次皆不精確，inexact=1。

## Binary32 round-to-nearest-even 與五種 FP 例外

**Tie：$1+2^{-24}$**
GRS=100，保留 LSB=0
candidate：1.0
inexact

**Above half：$1+3\cdot 2^{-25}$**
GRS=110
round up：$1+2^{-23}$
inexact

**Underflow tie：$2^{-150}$**
0 與 $2^{-149}$ 的中點
even→+0
underflow + inexact

**Overflow：max+max**
exact result > max finite
rounded→+∞
overflow + inexact

**Invalid：0×∞、∞−∞、0/0 → quiet NaN；1/±0 → signed infinity。**

G/R/S 是 guard、round、sticky；tie 選最低保留 bit 為 0，NaN 依 payload 與 comparison 規則傳播。

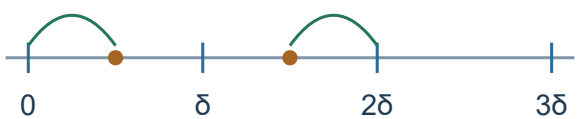
這裡的 $2^{-24}$ 是相鄰值間距的一半；不是「精確和的一半」。FMA 對精確 ab+c 只捨入一次，分開的乘法與加法各自捨入，結果可能不同；本章稍後給出可核對的位元數例。

# Subnormal：範圍延伸與旗標要分開判斷

令 $\delta=2^{-149}$。binary32 的正 subnormal 為 $F\delta$，其中 fraction 整數 $1\le F\le 2^{23}-1$；最小 normal 為 $2^{23}\delta=2^{-126}$。接近零的間距固定為 $\delta$，跨到最小 normal 時仍連續。

**binary32: $\delta = 2^{-149}$; RNE; gradual underflow**

**靠近 0**




**$\delta/2 \to 0$; $3\delta/2 \to 2\delta$**

underflow=1, inexact=1

精確 δ：兩個旗標都為 0

**靠近最小 normal**




0x007fffff = $(2^{23}-1)\delta$

**0x00800000 = $2^{23}\delta$**

跨邊界的間距仍為 δ

右圖第一點的 index 是 $2^{23}-2$；兩個視窗之間不採共同原點。

**Subnormal 不等於一定設定 underflow 旗標。** 精確的 $2^{-126}/2$ 得 0x00400000，沒有誤差；圖中兩個中點則因 tiny 且 inexact，同時設定兩個旗標。下一頁明定浮點環境並列出比較。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

## 用同一張表核對 subnormal 的精確性

本表固定 RNE、gradual underflow、停用 trap、旗標初值為 0，並在捨入後判定 tininess。所有輸入皆能以 binary32 精確表示；UF、NX 分別代表 underflow 與 inexact 旗標。

| 運算 | 精確值 | RNE 結果 bits | UF | NX |
|---|---|---|---|---|
| $2^{-126} / 2$ | $2^{-127}$ | 0x00400000 | 0 | 0 |
| $\delta / 2$ | $\delta/2$ | 0x00000000 | 1 | 1 |
| $(3\delta) / 2$ | $3\delta/2$ | 0x00000002 | 1 | 1 |
| 最小 normal − 最大 subnormal | $\delta$ | 0x00000001 | 0 | 0 |

前兩個中點以 fraction 整數的奇偶選擇：0 與 1 之間選 0；1 與 2 之間選 2。旗標記錄本次運算條件，不儲存誤差大小。若開啟 flush-to-zero（FTZ）或把 subnormal 輸入當成 0，已改變本節模型，不能直接套用此表。

表示法與 gradual underflow 可參照 Oracle Numerical Computation Guide §2.3。

## 特殊值：同時寫出結果與例外

最大有限值加自身在 RNE 下得 +∞，設定 overflow 與 inexact。0×∞、∞−∞、0/0 為 invalid，停用 trap 時回傳 quiet NaN。

有限非零數除以 ±0 得帶符號的 infinity，符號由兩輸入的 sign bits XOR 決定，設定 divide-by-zero；∞/+0 則仍為 infinity，不因這次運算設定 divide-by-zero。詳見 Oracle Numerical Computation Guide 的例外表。

NaN 的比較須依 predicate 判斷：普通相等比較即使兩邊都是同一個 NaN 也為 false，不等比較為 true；ordered 與 unordered 不能混用。+0 與 −0 比較相等，但倒數與 signbit 可區分。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 浮點乘法

對 $a = (-1)^{s_a} 1.F_a 2^{e_a}$、$b = (-1)^{s_b} 1.F_b 2^{e_b}$，exact significand product 是

$$(1.F_a)(1.F_b),$$

result exponent 概念上是 $e_a + e_b$。Product 可落在 $[1, 4)$，需 normalize，再捨入到 target precision。

Sign 由 $s_a \oplus s_b$ 決定。Subnormal operands 可能需先 normalize 成有效 precision，或因 tiny result 進入 underflow path。

Zero multiplication 要處理 signed zero 與 infinity combinations。$0 \times \infty$ 是 invalid operation，不是一般 zero。

乘法比加法少 alignment，但 significand multiplier 可能較寬，仍需 extra precision 與 final rounding。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# MIPS 浮點指令

本章案例 MIPS floating-point 使用 coprocessor 1 registers，例如 `lwc1`、`swc1` 搬移 floating-point words，`add.s`、`add.d` 區分 formats。

Floating-point comparison 設定 condition code，branch 透過 `bc1t`、`bc1f` 等指令檢查。NaN 的 ordered/unordered comparison 需要特別 semantics。

Register number 空間與 general-purpose registers 分開。Compiler 要避免 format 混用並計算 save area 大小。

本節是教材 MIPS 案例。不要把 MIPS FP encoding 直接套用到 RISC-V 或 Arm。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 精確算術與捨入模式

Round-to-nearest-even 是常用 default，因為它降低 bias。Directed modes 可往正 infinity、負 infinity 或 zero 捨入，適合 interval arithmetic 或 precise bounds。

ULP 是相鄰 representable numbers 的間距。絕對 error 小不代表 relative error 小；在很大 exponent 下，ULP 也可以很大。

Floating-point format 的 precision 是 significand bits，dynamic range 由 exponent bits 決定。增加 exponent 不增加 precision，增加 fraction 才增加 precision。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## NaN、signed zero 與比較

NaN 不與任何 value 成立 ordinary ordered equality，包含自己。Program 可用 `isnan` 或 unordered comparison 測量。

$+0 = -0$ 在 IEEE equality 中成立，但兩者 sign bit 不同，可能在 reciprocal、copy sign 與 serialization 中顯現。

Infinity arithmetic 有部分代數規則。$\infty - \infty$、$0 \times \infty$、$\infty / \infty$ 是 invalid，結果通常為 quiet NaN。

比較浮點值不能只用 signed integer bit order，因為 negative numbers 的 encoding 與 NaN。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.6 平行與結合律

Exact integer addition 的 associativity 在 modular arithmetic 中也成立，但 floating-point rounding 使

$$(a + b) + c \neq a + (b + c)$$

可能成立。即使 exact sum 相同，rounding point 不同也會改變 result。

Parallel reduction 若改變 summation tree，結果可能因 rounding 改變。這不是 race，而是 numerical reproducibility 問題。

若要求bitwise reproducibility，需固定求和次序及相同浮點環境（格式、捨入模式、中間精度與FMA收縮等），或使用具明確順序無關保證的reproducible accumulator。Compensated summation與提高precision主要改善誤差，本身不保證換序後位元相同。

標準Kahan若每次基本運算都以binary32的round-to-nearest-even執行，順序 $[2^{25}, 1, -2^{25}]$ 得0，改成 $[2^{25}, -2^{25}, 1]$ 得1。第一順序中加1時sum仍為 $2^{25}$、補償c變−1；接著 $-2^{25} + 1$ 位於中點，RNE又捨回 $-2^{25}$，總和成0。第二順序先精確抵銷成0，再加1得1。補償降低許多累加誤差，卻沒有消除對運算次序與環境的依賴。

同一個程式在不同 thread scheduling 下得到略不同答案，不一定是 memory corruption；先檢查 reduction order。

### FMA 與兩次 rounding 的差別

浮點 add 的特殊 grouping 也要用 bit pattern 驗證，而不是宣稱數學上相等。以 binary32 $x = 2^{24}$ 為例，$x$ 的 ULP 是 2。x+1 恰好落在 x 與 x+2 的中點，最低保留 bit 要求選 even，所以先 round 成 x：

$$((x + 1) - x) = 0, \qquad (x - x) + 1 = 1.$$

兩式在 exact real arithmetic 相等，在逐次 binary32 rounding 下不相等。這是 rounding point 改變，不是 data race。

令 $a = b = 1 + 2^{-23}$（`0x3f800001`）、$c = -(1 + 2^{-22})$（`0xbf800002`）。分開執行 `mul.s` 後再 `add.s`：

$$(1 + 2^{-23})^2 = 1 + 2^{-22} + 2^{-46},$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

乘積先 round 成 $1 + 2^{-22}$，再加 $c$ 得 `+0`。若假設的 fused multiply-add（FMA）依 IEEE semantics 只 round 一次，exact $ab + c = 2^{-46}$ 可直接表示為 `0x28800000`。這不是因為 FMA 改變 real algebra，而是少了 intermediate `1+2^{-22}` rounding。

本章舊版 MIPS core 不保證有 FMA；有 `fma` 的 extension 必須另外指定 rounding mode 與 operand format。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.7 x86 浮點案例

教材比較 x87 與 SSE2。x87 以 internal extended-precision stack registers 執行，SSE2 使用 XMM registers 作 SIMD packed/scalar operations。

把 value 存在寬 internal precision，再 round 到 memory format，與每一步都 round 到 target precision，可能得到不同結果。ABI 與 compiler flags 會改變 intermediate precision。

SSE2 支援 packed multiply/add 等 instructions，可提高資料層級平行。Instruction form、alignment 與 lane semantics 必須符合實際 ISA。

歷史 x87 與今日 AVX 不可混用。Version、OS context 與 compiler `fast-math` 都要記錄。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.8 算術誤解

「Floating point 只是近似，所以不用管」忽略近似方式本身有嚴格規則。IEEE result、exception flags 與 rounding mode 能被應用精確依賴。

「Fast-math 只會提高速度」忽略 compiler 可重排、假設無 NaN 或改變 associativity。安全性與 reproducibility 可能受影響。

「Division 一定比 multiplication 慢固定倍數」忽略 operand-dependent divider、pipeline、specialization 與 instruction mix。

每條誤解都要求一個可驗證的基準與反例。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 3.9 算術小結

Integer operations 的 width、overflow 與 modulo semantics 要一起看。Multiply/divide 的 iteration 可能是 32 次數量級，但實際 cycles 依 radix、normalization 與 pipeline 不同。

Floating-point unit 是三個 functions 的組合：representation、arithmetic、rounding/exception。Add 的關鍵是 alignment 與 lost bits；multiply 的關鍵是 significand product 與 normalization。

精確性驗收應包含 normal、zero、subnormal、infinity、NaN、overflow、underflow 與 tie-to-even。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 數值格式演進

早期 machines 使用不同 exponent width、bias 與 signed representations。IEEE 754 標準化 binary formats、operation rounding 與 special values，使程式能跨硬體得到可預測結果。

硬體從 serial add/shift multiplier 走向 radix recoding、parallel partial-product tree、pipeline FP unit 與 SIMD。Instruction 與 compiler 也共同把更多算術工作放到 vector lanes。

演進沒有消除數值分析。大動態範圍、ill-conditioned problems 與 reproducible parallelism 仍需 algorithm-level 處理。

將 format、arithmetic hardware、rounding 與 algorithm 分成四層，不要用「IEEE 已解決浮點」概括。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 章末驗收

學生要能以 bit-level trace 完成以下工作。

1. 判定 signed/unsigned addition overflow，並檢查 carry-in/carry-out 與 sign-bit 條件。
2. 用 4-bit 教學模型逐回合追蹤 multiply 與 restore division，說明 register width、carry 與 correctness invariant。
3. 編碼與解碼 binary32，正確處理 subnormal、infinity、NaN 與 signed zero。
4. 在 alignment 後使用 guard/round/sticky 完成 FP add，並比較 tie-to-even 與 above-half 的 rounding。
5. 對 overflow、underflow、invalid、divide-by-zero 與 inexact 給出 result 和 flag。
6. 證明 exact-equal algebraic expressions 在 floating point 不一定得到同一 bit pattern。
7. 比較 FMA 與 multiply-then-add 的 rounding point，求出可觀察差異的 bit patterns。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# 參考資料

Patterson 與 Hennessy，*Computer Organization and Design: The Hardware/Software Interface*，Revised Fourth Edition，3.1–3.10，印刷頁 222–283（PDF 頁 249–310）。

整數乘法、restoring division 與 multi-step floating-point operation 的成本以本文宣告的教學模型計算，不代表特定商用 core 的固定 latency。IEEE 754 binary32 special values、rounding modes、guard/round/sticky 與 FMA single-rounding 條件均需與 operand representation 一起解讀。

完整書目、版本與未交付範圍見 章節導覽。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# 參考資料與署名

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study